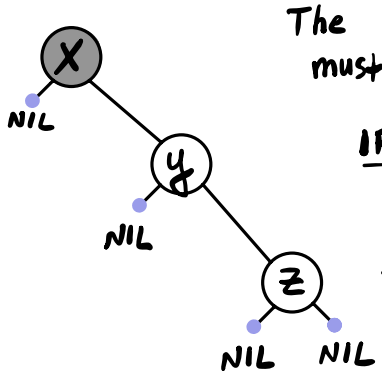
Red-black Tree Homework

Problem 1. It is not possible to color all binary search trees so that they are **red-black** trees. Give an example of a binary search tree that cannot be colored to be a **red-black** tree and explain why.



The root must be black so the path $X \to NIL$ must have 2 black nodes.

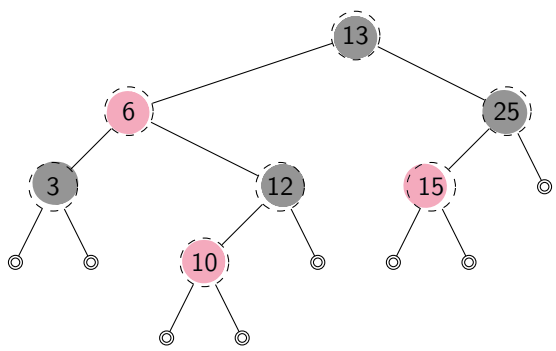
IF y IS RED: then z must be black, so $X \to y \to z \to NIL$ has 3 black nodes. X

IF y IS BLACK: then $X \to y \to NIL$ has 3 black nodes. X

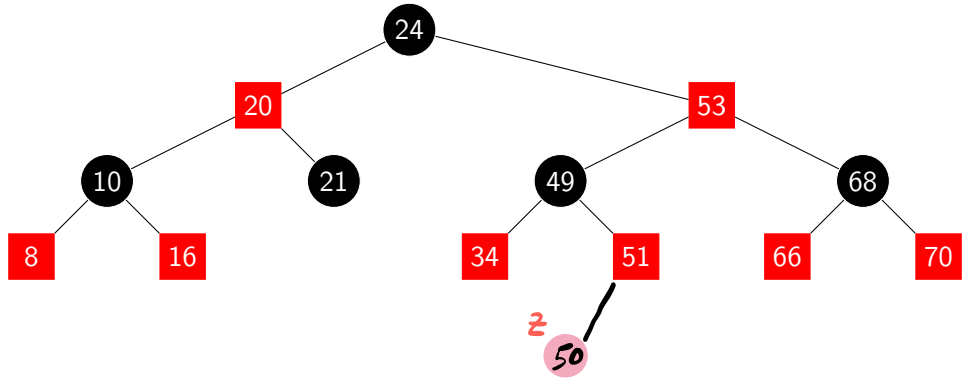
Thus, this tree cannot be colored!

Alternate explanation: If T is an RBT, then $h \leq 2\log_2(n+1)$. The contrapositive of this statement is If $h > 2\log_2(n+1)$, then T is not a RBT.
Consider a tree with $n = 7$ nodes in a line. $2\log_2(8) = 2 \cdot 3 = 6$ and $h = 8 > 6$.

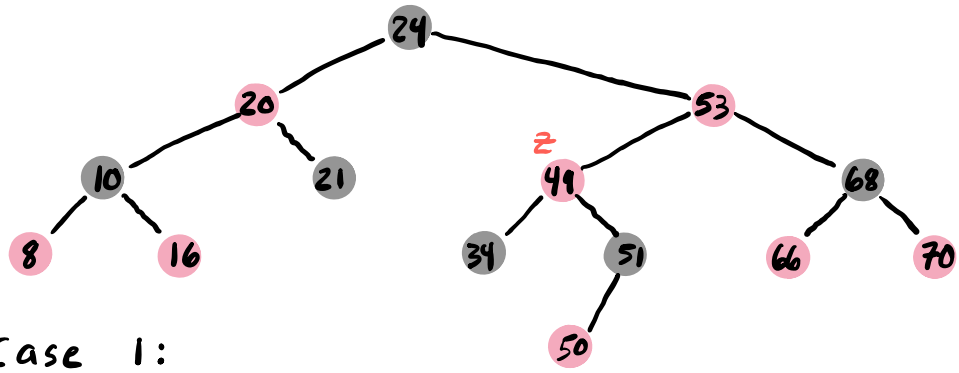
Problem 2. Color the nodes of the following binary search tree so that it is a **red-black** tree.



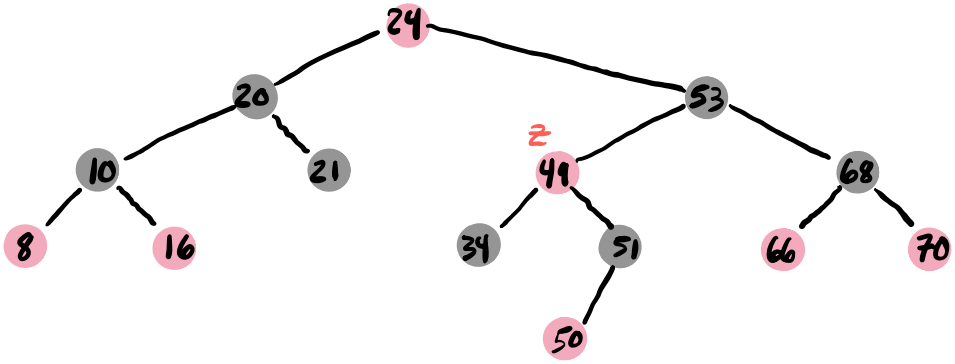
Problem 3. Draw the result of applying `RBTreeInsert` to insert 50 in the following **red-black** tree. Indicate which nodes are red or black in your drawing. Be sure that the resulting tree has all the properties of a **red-black** tree. Show your work, i.e., show the effects of the different algorithm steps on the tree.



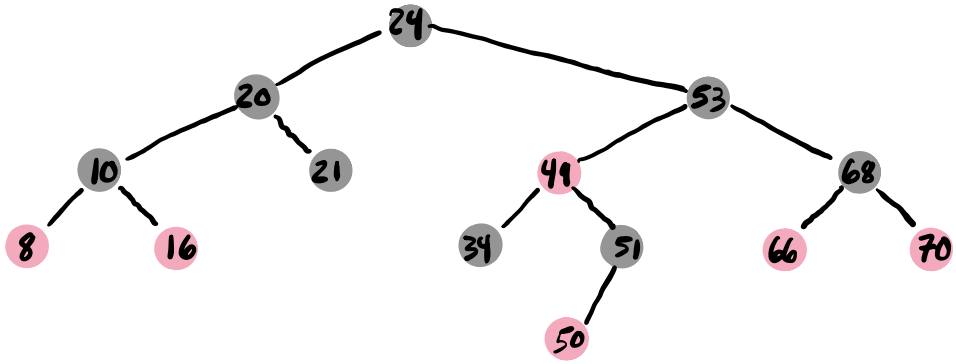
Case 1:



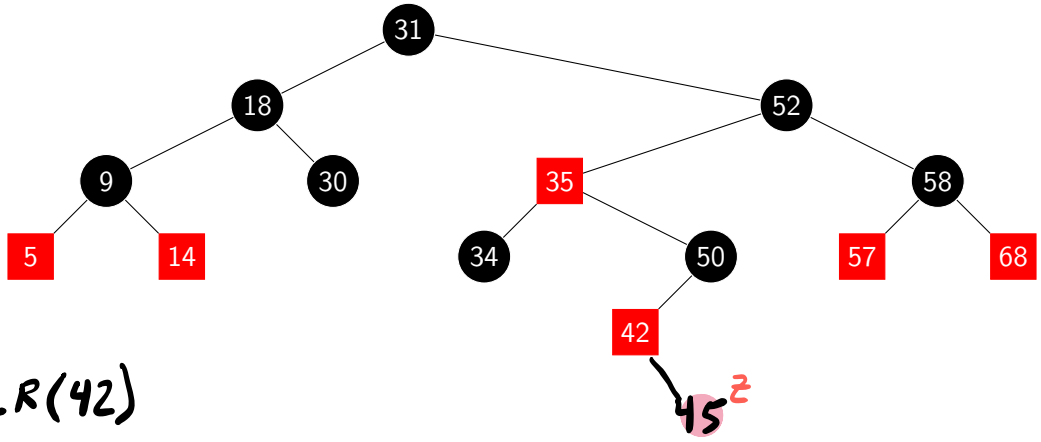
Case 1:



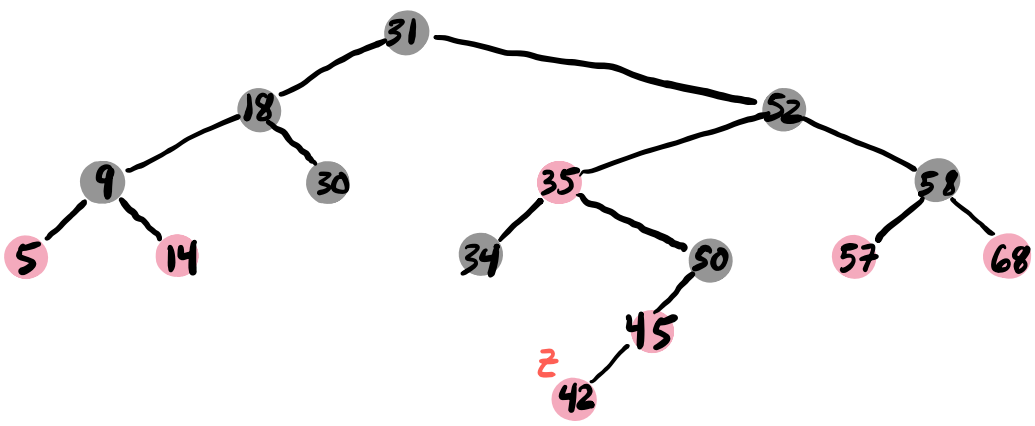
Recolor root:



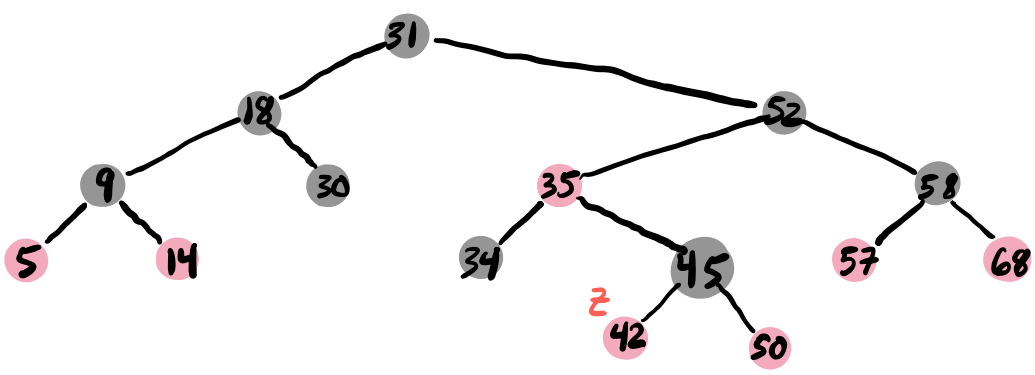
Problem 4. Draw the result of applying `RBTreeInsert` to insert 45 in the following **red-black** tree. Indicate which nodes are red or black in your drawing. Be sure that the resulting tree has all the properties of a **red-black** tree. Show your work, i.e., show the effects of the different algorithm steps on the tree.



Case 2: LR(42)



Case 3: RR(50) + Recolor



Problem 5. Another way to create a balanced binary search tree is an AVL-tree; we augment the tree so that for every node v in the tree:

- v has a field $v.\texttt{height}$ that stores the height of the subtree rooted at v.

Consider the tree below and

(a) Label the height of every node in the tree.

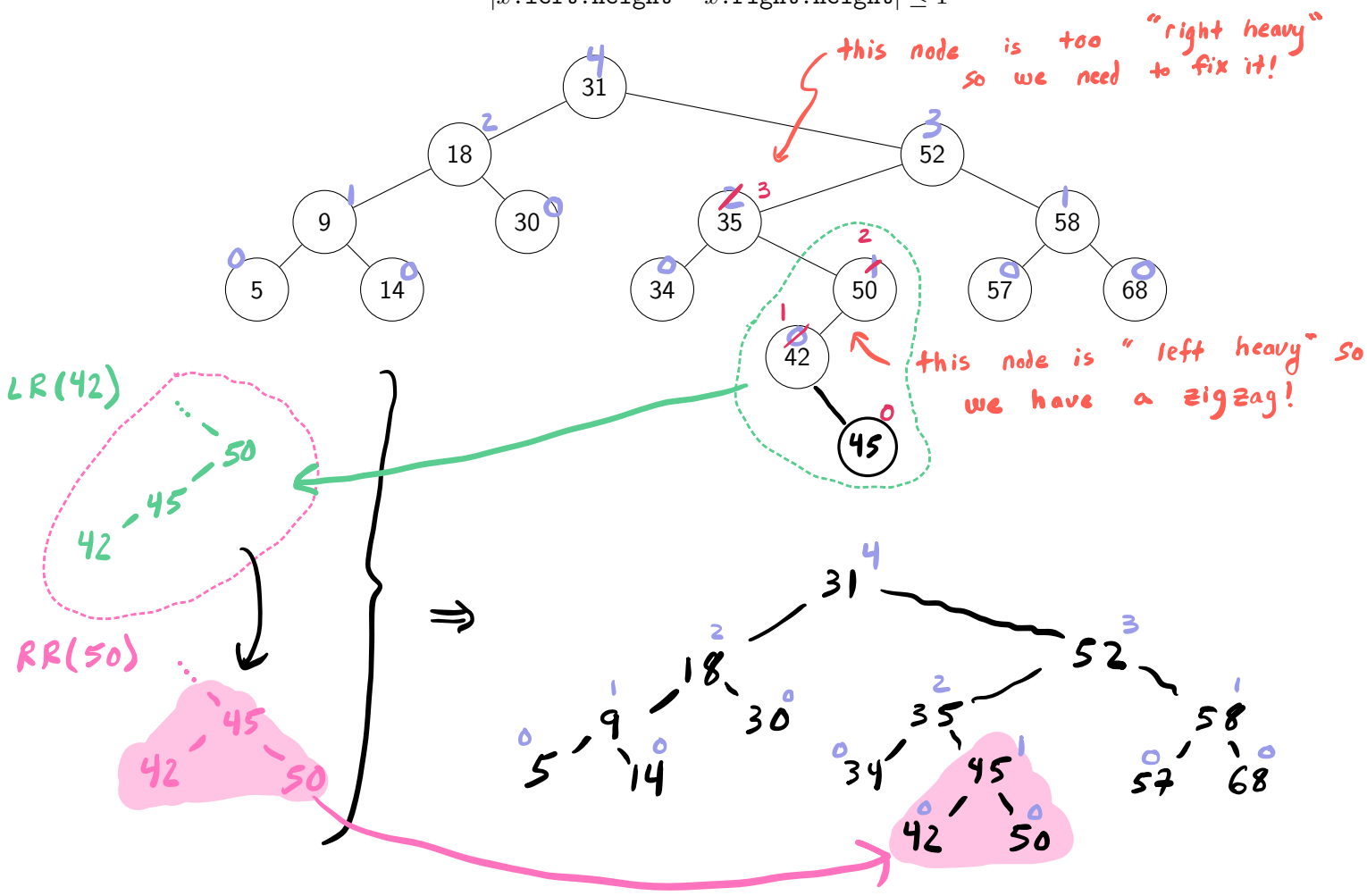
(b) Insert 45 into the binary search tree.

(c) Beginning at 45, update the height of the ancestors of 45 until you find a node where

$$|x.\texttt{left.height} - x.\texttt{right.height}| > 1$$

Once this node is found, you can now determine what node(s) must be rotated to fix the height of the tree. Rotate the nodes of the tree so that after the rotation(s) are finished, every node of the tree has

$$|x.\texttt{left.height} - x.\texttt{right.height}| \leq 1$$



If you can generalize this idea, then you can invent an AVL-tree! :)